

Deep Q-Learning

Reinforcement Learning

Christian Igel

Department of Computer Science

🐦 @christian_igel

UNIVERSITY OF COPENHAGEN



Outline

Function approximators

Learning value functions from data

Deep Q-Learning

Further stabilizing the learning

Outline

Function approximators

Learning value functions from data

Deep Q-Learning

Further stabilizing the learning

Function approximators

- Value functions represented as tables are very limited:
 - Not suitable for continuous state and/or action spaces
 - No generalization (learning about unseen states or state-action pairs from similar states or state-action pairs)
- We can use function approximators with parameters w for the state value function

$$\hat{V} : S \rightarrow \mathbb{R}$$

or the state-action value function

$$\hat{Q} : S \times A \rightarrow \mathbb{R}$$

Examples of function approximators

- Look-up tables (as in tabular Q -learning) can be viewed as special cases of function approximators that do not generalize.
- For example, assume a feature mapping $\phi : S \rightarrow \mathbb{R}^d$ and discrete actions $A = \{a_1, \dots, a_K\}$, we could have linear approximators

$$\hat{Q}(s, a_i) = \hat{Q}_i(s) = \mathbf{w}_i^\top \phi(s)$$

with $\mathbf{w}_i^\top \in \mathbb{R}^d$ for each $i = 1, \dots, K$.

- If the function approximators are deep neural networks, we talk of Deep RL.

Why deep neural networks (DNNs)?

Deep neural networks as function approximators

- can learn highly non-linear functions, which allows them to model strong nonlinear interactions, represent multimodal functions, and approximate discontinuities;
- have proven to be applicable to large input spaces;
- ideally learn high-level representations (e.g., of goals and threats) from low-level (sensory) input;
- allow to incorporate prior knowledge in the network architecture (CNNs, RNNs, ...);
- allow iterative optimization using stochastic gradient descent.

Outline

Function approximators

Learning value functions from data

Deep Q-Learning

Further stabilizing the learning

Learning value functions from data

We learn the value function approximators from data

$$\mathcal{D} = \{((s_{t_1}, a_{t_1}), y_{t_1}), ((s_{t_2}, a_{t_2}), y_{t_2}), \dots\} ,$$

as in supervised learning, e.g., minimizing $\ell(y, y') = \frac{1}{2}(y - y')^2$:

$$E_{\mathcal{D}}(\mathbf{w}) = \frac{1}{|\mathcal{D}|} \sum_{((s,a),y) \in \mathcal{D}} \ell(\hat{Q}(s,a) - y) = \frac{1}{2|\mathcal{D}|} \sum_{((s,a),y) \in \mathcal{D}} (y, \hat{Q}(s,a))^2$$

But what should be the targets y ?

Bellman error

Bellman error (difference between RHS and LHS in Bellman equation for approximate value functions):

$$\delta_{\text{BE}}^{\pi}(s, a) = \sum_{s' \in S} P_{s, s'}^a \left(R_{ss'}^a + \gamma \sum_{a' \in A} \pi(s', a') \hat{Q}(s', a') \right) - \hat{Q}(s, a)$$

Estimating the Bellman error from interactions following π gives the TD error:

$$\delta_{\text{TD}}^{\pi}(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}) = \underbrace{r_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1})}_{y_t} - \hat{Q}(s_t, a_t) \quad ,$$

TD error based on the Bellman optimality condition:

$$\delta_{\text{TD}}^*(s_t, a_t, r_{t+1}, s_{t+1}) = \underbrace{r_{t+1} + \gamma \max_{a' \in A} \hat{Q}(s_{t+1}, a')}_{y_t} - \hat{Q}(s_t, a_t)$$

Gradient

- To get zero TD errors, we minimize the (halved) squared TD errors.
- Let us consider $y_t = r_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1})$ or
 $y_t = r_{t+1} + \gamma \max_{a' \in A} \hat{Q}(s_{t+1}, a')$.
- Then $\frac{1}{2}(\delta_{\text{TD}}^\pi(s_t, a_t, r_{t+1}, s_{t+1}))^2 = \frac{1}{2}(y_t - \hat{Q}(s_t, a_t))^2$ and

$$\begin{aligned}\nabla_{\mathbf{w}} \frac{1}{2}(\delta_{\text{TD}}^*(s_t, a_t, r_{t+1}, s_{t+1}))^2 &= \nabla_{\mathbf{w}} \frac{1}{2} (y_t - \hat{Q}(s, a))^2 \\ &= - (y_t - \hat{Q}(s, a)) \nabla_{\mathbf{w}} \hat{Q}(s, a) \\ &= (\hat{Q}(s, a) - y_t) \nabla_{\mathbf{w}} \hat{Q}(s, a)\end{aligned}$$

under the assumption that $\nabla_{\mathbf{w}} y_t = 0$.

Gradient step

- Each gradient step corresponds to a step in generalized policy iteration.
- Steepest descent learning rule:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \underbrace{\frac{1}{|\mathcal{D}|} \sum_{((s,a),y) \in \mathcal{D}} \overbrace{(y - \hat{Q}(s,a))}^{\text{TD error}} \nabla_{\mathbf{w}} \hat{Q}(s,a)}_{-\nabla_{\mathbf{w}} E_{\mathcal{D}}(\mathbf{w})}$$

with learning rate $\alpha > 0$.

Relation to tabular Q -learning

- Sanity check: discrete A and S and a table with $|A| \times |S|$ entries corresponding to $|A| \times |S|$ -dimensional parameter vector $\mathbf{w}$ with $\hat{Q}(s, a) = w_{s+|S| \cdot a}$.
- For $((s_t, a_t), y_t)$ we have

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha (y_t - \hat{Q}(s_t, a_t)) \nabla_{\mathbf{w}} \hat{Q}(s_t, a_t) .$$

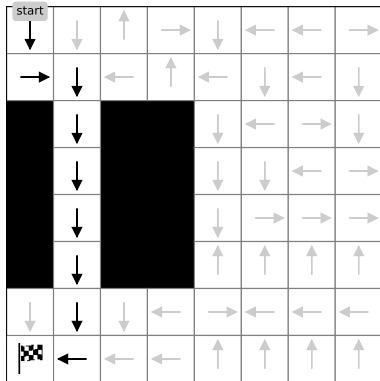
- Tabular function approximator, $[\nabla_{\mathbf{w}} \hat{Q}(s, a)]_i$ is one for $i = s_t + |S| \cdot a_t$ corresponding to (s_t, a_t) and zero otherwise.
- We can replace $w_{s+|S| \cdot a}$ by $\hat{Q}(s, a)$ and recover SARSA

$$\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + \alpha \underbrace{[r_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1})]}_{y_t} - \hat{Q}(s_t, a_t)$$

and Q -learning:

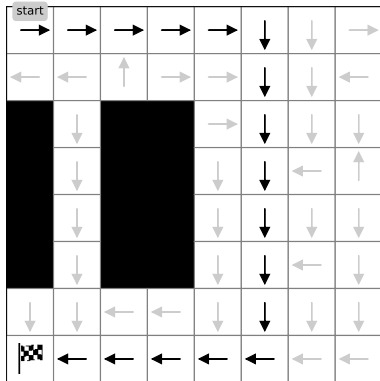
$$\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, \text{off-policy } a_t) + \alpha \underbrace{[r_{t+1} + \gamma \max_a \hat{Q}(s_{t+1}, a)]}_{y_t} - \hat{Q}(s_t, a_t)$$

Recall: On-policy vs. off-policy



start	-9.00	-8.00	-7.85	-7.43	-6.95	-6.63	-6.42	-6.35
-8.00	-7.00	-7.82	-7.32	-6.76	-6.48	-6.30	-6.26	
█	-6.00	█	█	█	-6.24	-6.17	-6.09	-6.05
	-5.00				-5.82	-5.80	-5.79	-5.75
	-4.00				-5.38	-5.41	-5.45	-5.48
	-3.00				-4.83	-5.01	-5.17	-5.25
-1.00	-2.00	-2.90	-3.50	-4.21	-4.62	-4.89	-5.01	
flag	-1.00	-2.00	-2.98	-3.82	-4.43	-4.81	-4.93	

On-policy vs. off-policy



start	-19.00	-17.75	-16.72	-15.19	-14.04	-12.93	-12.58	-12.11
	-18.03	-17.96	-17.31	-13.91	-12.92	-11.89	-11.94	-11.62
		-11.46			-11.70	-10.79	-11.16	-10.97
		-6.43			-9.75	-9.70	-10.33	-10.26
		-4.06			-8.72	-8.63	-9.42	-9.59
		-3.01			-6.46	-7.47	-8.50	-8.82
	-1.00	-2.11	-3.08	-3.98	-5.50	-6.48	-7.57	-8.06
	flag	-1.00	-2.17	-3.21	-4.29	-5.53	-6.56	-7.44

But ...

- There is a problem in the derivation above: We assumed that y_t is independent of w !
- This is not true in general for $y_t = r_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1})$ and $y_t = r_{t+1} + \gamma \max_{a' \in A} \hat{Q}(s_{t+1}, a')$.
- In deep RL we in general do not consider a dependency of y_t on w , which can be justified when introducing a target network.

Dealing with terminal states

- When a state s_t is terminal, there are not rewards for time steps $t' > t$ and value function estimates for time steps $t'' > t$ are set to zero.
- Example: TD error $\leftarrow \begin{cases} r_{t+1} - \hat{V}(s_t) & \text{if } s_{t+1} \text{ terminal} \\ r_{t+1} + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t) & \text{else} \end{cases}$
- Often it is modelled that the agent also observes an indicator variable

$$d_t = \begin{cases} 1 & \text{if } s_t \text{ terminal} \\ 0 & \text{otherwise} \end{cases}$$

and the example above can be written as:

$$\delta \leftarrow r_{t+1} + (1 - d_{t+1})\gamma \hat{V}(s_{t+1}) - \hat{V}(s_t)$$

- Does not change our MDP definition, think of d_t being part of the state.

Outline

Function approximators

Learning value functions from data

Deep Q-Learning

Further stabilizing the learning

Function approximators for discrete actions

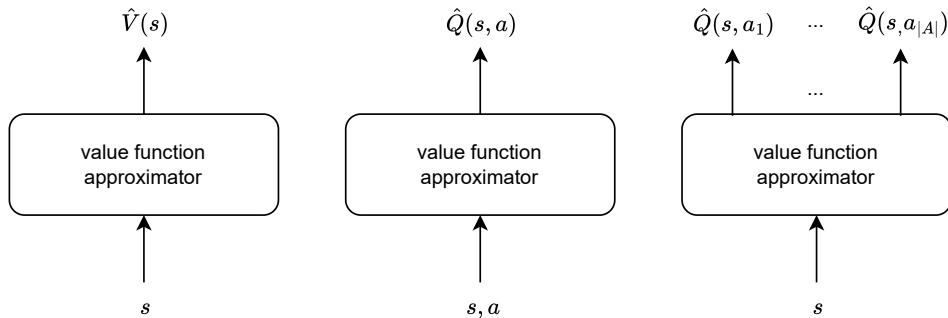
- Assume discrete actions $A = \{a_1, \dots, a_K\}$.
- Instead of a mapping $S \times A \rightarrow \mathbb{R}$ we can learn

$$\hat{Q} : S \mapsto \mathbb{R}^{|A|}$$

approximating $Q(s, a_i)$ by $[\hat{Q}(s)]_i$.

- In the following, we do not distinguish between $\hat{Q}(s, a_i)$ and $[\hat{Q}(s)]_i$ and set $\max_a \hat{Q}(s, a) \rightarrow 0$ if s is a terminal state.
- If $\hat{Q}$ is a deep neural network, we talk about deep Q -learning.

Value function approximators



Variance reduction and experience replay

- Updating a neural network using stochastic gradient step based on single observation is not recommended because the variance is too high
- **Solution 1:** Accumulate the gradients and perform an update after T_{update} steps; easy to parallelize (Mnih et al., 2016)
- **Solution 2:** Store experiences $\langle s, a, r, s' \rangle$ in a finite FIFO buffer and sample mini-batches from this buffer for learning (Riedmiller 2005, Mnih et al., 2015), if buffer is full, the oldest element will be removed if a new one is added

Experience replay buffer

- Storing experiences in a buffer allows the learning algorithm to use (potentially costly) observations several times.
- In supervised learning, neural networks typically also need several training epochs to converge.
- In RL, the optimization problem changes when the policy changes (e.g., because $\hat{Q}$ changes).
- It is important to randomly sample the mini-batches from a larger buffer. Simply combining the last observations in a mini-batch $\mathcal{D}$ is not a good strategy because then the elements in the mini-batch may be too similar to each other because they typically stem from the same episode, which may lead to a bias in the gradient estimation.

Sketch of simple mini-batch Q-learning

Algorithm: Simple Deep Q-learning with experience replay

```
1 foreach episode do
2   initialize  $s \sim p_{\text{start}}$ 
3   repeat
4     choose  $a$  based on  $\hat{Q}$  // e.g.,  $\epsilon$ -greedy
5     take action  $a$ , observe  $r, s', d$ 
6     store  $\langle s, a, r, s', d \rangle$  in  $M$ 
7     sample batch  $\mathcal{D} = \{ \langle s_i, a_i, r_i, s'_i, d_i \rangle \mid i = 1, \dots, n_{\text{batch}} \}$  from  $M$ 
8     set targets  $\forall i : y_i = r_i + (1 - d)\gamma \max_{a'} \hat{Q}(s'_i, a')$ 
9     update network  $\mathbf{w} \leftarrow \mathbf{w} - \alpha \sum_{i=1}^{n_{\text{batch}}} \nabla_{\mathbf{w}} \ell(y_i, \hat{Q}(s_i, a_i))$ 
10     $s \leftarrow s'$ 
11  until  $s$  is terminal
```

Recall: Greedy and soft policy

- Greedy policy:

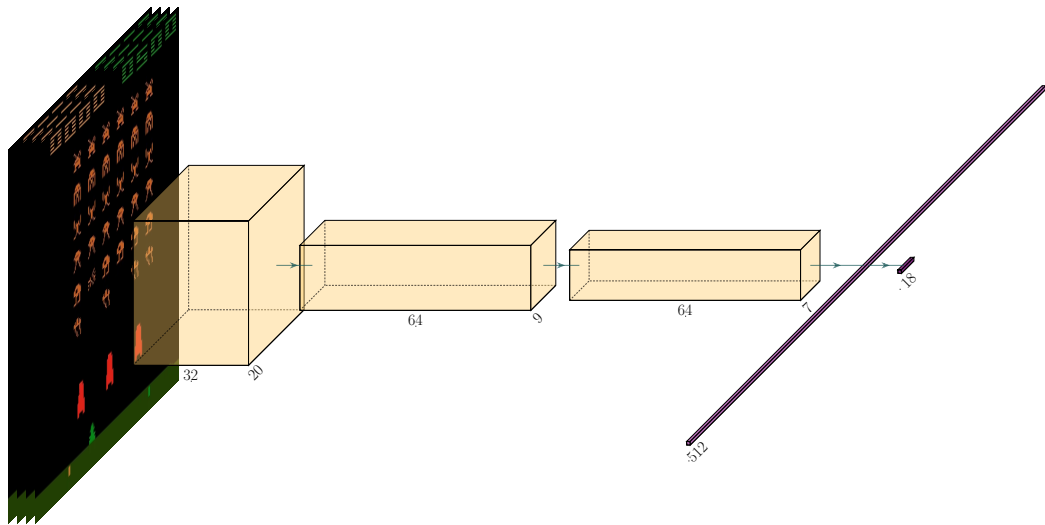
$$\pi(s) = \operatorname{argmax}_{a \in A} Q(s, a)$$

- There is need for exploration!
- Soft policies: $\pi(s, a) > 0$ for all s and a
- Example: ϵ -soft (ϵ -greedy) policy, which follow the greedy policy but take a random action with a probability of ϵ at every step:

$$\text{probability for actions:} \quad \begin{array}{ll} \frac{\epsilon}{|A|} & 1 - \epsilon + \frac{\epsilon}{|A|} \\ \text{non-max} & \text{greedy} \end{array}$$

Exploration can be adjusted during learning, e.g., by reducing ϵ over time.

ATARI example



Delayed Q update

- Updating the $\hat{Q}$ network changes also the targets for updating the $\hat{Q}$ network, which leads to instabilities
- Idea: Use a **different target network** for computing the temporal difference errors and update the target network on a **slower time scale** (Mnih et al., 2015, 2016)
- Same network architecture is used, only the parameters vary: we distinguish between w and w_{target}
- Same for policy parameters: θ and θ_{target}
- Two ways for **delayed update**:
 - Copy parameters (e.g., $w_{\text{target}} \leftarrow w$) every $T_{\text{target-update}}$ steps (Mnih et al., 2016)
 - Smoothly blend the parameters (e.g., $\theta_{\text{target}} \leftarrow \rho \theta_{\text{target}} + (1 - \rho) \theta$ for $\rho \in]0, 1[$ → Polyak averaging)

Deep Q-learning (DQN)

Algorithm: Deep Q-learning (DQN)

```

1 foreach episode  $e = 1, \dots$  do
2   initialize  $s \sim p_{\text{start}}$ 
3   repeat
4      $\epsilon = \epsilon_{\text{stop}} + (\epsilon_{\text{start}} - \epsilon_{\text{stop}})e^{-\beta_{\epsilon}(e-1)}$ 
5     choose  $a$  based on  $\hat{Q}$   $\epsilon$ -greedy; take action  $a$ , observe  $r, s', d$ 
6     store  $\langle s, a, r, s', d \rangle$  in  $M$ 
7     sample  $\mathcal{D} = \{ \langle s_i, a_i, r_i, s'_i, d_i \rangle \mid i = 1, \dots, n_{\text{batch}} \}$  from  $M$ 
8     set targets  $\forall i : y_i = r_i + (1 - d)\gamma \max_{a'} \hat{Q}_{\text{target}}(s'_i, a')$ 
9     compute loss  $E_{\mathcal{D}}(\mathbf{w}) = \sum_{i=1}^{n_{\text{batch}}} \ell(y_i, \hat{Q}(s_i, a_i))$ 
10    update  $\mathbf{w}$  based on  $\nabla_{\mathbf{w}} E_{\mathcal{D}}(\mathbf{w})$  (e.g., perform AdamW step)
11    update target network  $\mathbf{w}_{\text{target}} \leftarrow \rho \mathbf{w}_{\text{target}} + (1 - \rho) \mathbf{w}$ 
12     $s \leftarrow s'$ 
13  until  $s$  is terminal

```

Overestimation bias

- Consider n_A random variables $X_1, \dots, X_{n_A}$. We are interested in $\max_i \mathbb{E}[X_i]$.
- For each X_i , we have n_i i.i.d. realizations $\{x_{i,1}, \dots, x_{i,n_i}\}$ with average $\hat{x}_i$.
- From Jensen's inequality, we know $\max_i \mathbb{E}[X_i] = \max_i \mathbb{E}[\hat{x}_i] \leq \mathbb{E}[\max_i \hat{x}_i]$.

That is, $\max_i \hat{x}_i$ tends to overestimate $\max_i \mathbb{E}[X_i]$.

- We can address this issue by maintaining two i.i.d. samples from each X_i with means $\hat{x}_i$ and $\hat{x}'_i$.

If we use the first one to determine $i^* = \arg \max_i \hat{x}_i$, then $\hat{x}'_{i^*}$ is an unbiased estimate of $\max_i \mathbb{E}[X_i]$.

Double Q-learning

- Q -learning uses a max operation $\rightarrow$ overestimation bias
- In deep Q -learning with delayed Q -update, we already keep track of two value function approximators, the online network $\hat{Q}$ and the target network $\hat{Q}_{\text{target}}$.
- We exploit this in the double deep Q -learning network (DDQN), where the action is selected by the online network

$$a^* = \arg \max_a \hat{Q}(s', a) ,$$

but the target network evaluates it:

$$y = r + \gamma \hat{Q}_{\text{target}}(s', a^*)$$

- This DDQN approach is very common.

Double Deep Q-learning (DDQN)

Algorithm: Double Deep Q-learning (DDQN)

```

1 foreach episode  $e = 1, \dots$  do
2   initialize  $s \sim p_{\text{start}}$ 
3   repeat
4      $\epsilon = \epsilon_{\text{stop}} + (\epsilon_{\text{start}} - \epsilon_{\text{stop}})e^{-\beta_{\epsilon}(e-1)}$ 
5     choose  $a$  based on  $\hat{Q}$   $\epsilon$ -greedy; take action  $a$ , observe  $r, s', d$ 
6     store  $\langle s, a, r, s', d \rangle$  in  $M$ 
7     sample  $\mathcal{D} = \{ \langle s_i, a_i, r_i, s'_i, d_i \rangle \mid i = 1, \dots, n_{\text{batch}} \}$  from  $M$ 
8     set targets  $\forall i : y_i = r_i + (1 - d)\gamma \hat{Q}_{\text{target}}(s'_i, \arg \max_{a'} \hat{Q}(s', a'))$ 
9     compute loss  $E_{\mathcal{D}}(\mathbf{w}) = \sum_{i=1}^{n_{\text{batch}}} \ell(y_i, \hat{Q}(s_i, a_i))$ 
10    update  $\mathbf{w}$  based on  $\nabla_{\mathbf{w}} E_{\mathcal{D}}(\mathbf{w})$  (e.g., perform AdamW step)
11    update target network  $\mathbf{w}_{\text{target}} \leftarrow \rho \mathbf{w}_{\text{target}} + (1 - \rho) \mathbf{w}$ 
12     $s \leftarrow s'$ 
13  until  $s$  is terminal

```

Outline

Function approximators

Learning value functions from data

Deep Q-Learning

Further stabilizing the learning

Clipping

Limiting the impact of a single update step can be achieved by clipping:

$$\text{clip}(x, x_{\text{low}}, x_{\text{high}}) = \min(x_{\text{high}}, \max(x, x_{\text{low}})) = \begin{cases} x_{\text{high}} & \text{if } x > x_{\text{high}} \\ x_{\text{low}} & \text{if } x < x_{\text{low}} \\ x & \text{otherwise} \end{cases}.$$

One can either do gradient-clipping

$$\nabla_{w_i} \ell(y_{t+i}, \hat{Q}(a_{t+i}, s_{t+i})) \leftarrow \text{clip}(\nabla_{w_i} \ell(y_{t+i}, \hat{Q}(a_{t+i}, s_{t+i})), g_{\min}, g_{\max}) \quad \text{or}$$

$$\nabla_{\mathbf{w}} \ell(\dots) \leftarrow \nabla_{\mathbf{w}} \ell(y_{t+i}, \hat{Q}(a_{t+i}, s_{t+i})) \frac{\min\{g_{\max}, \|\nabla_{\mathbf{w}} \ell(y_{t+i}, \hat{Q}(a_{t+i}, s_{t+i}))\|\}}{\|\nabla_{\mathbf{w}} \ell(y_{t+i}, \hat{Q}(a_{t+i}, s_{t+i}))\|}$$

or clip the loss function directly:

$$\ell(y_{t+i}, \hat{Q}(a_{t+i}, s_{t+i})) \leftarrow \text{clip}(\ell(y_{t+i}, \hat{Q}(a_{t+i}, s_{t+i})), \ell_{\min}, \ell_{\max})$$

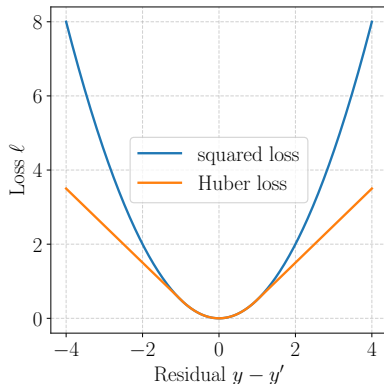
Changing the loss

- We can directly employ a robust loss function, for example, the Huber-loss:

$$\ell_{\text{Huber}}(y, y') = \begin{cases} \frac{1}{2}|y - y'|^2, & \text{if } |y - y'| \leq \delta, \\ \delta \left(|y - y'| - \frac{1}{2}\delta \right), & \text{otherwise} \end{cases}$$

with parameter $\delta > 0$.

- Training using weight-decay.



Rainbow

Rainbow (Hessel et al. , 2018) combines modifications of DQN into one algorithm:

- DDQN ($\rightarrow$ overestimation bias)
- Prioritized experience replay ($\rightarrow$ data efficiency)
- Multi-step bootstrapping, e.g., changing Q -learning target to:

$$\sum_{k=0}^{n_R-1} \gamma^k r_{t+k+1} + \gamma^{n_R} \max_{a'} \hat{Q}(s_{t+n_R}, a')$$

- Distributional Q -learning: Predict distribution of returns instead of expectation
- Network architecture changes: Noisy network layers to support exploration and dueling network (representing $\hat{Q}$ by $\hat{V}$ and advantage function)

Prioritized replay and multi-step learning were the most important features, followed by distributional Q -learning.

References

- C. Igel. *Concepts of Deep Reinforcement Learning: A Short Course*, chapter 4, 2026
- V. Mnih, et al. Human-level control through deep reinforcement learning. *Nature* 518:529–533, 2015
- V. Mnih et al. Asynchronous Methods for Deep Reinforcement Learning *International Conference on Machine Learning (ICML)*, 2016
- M. Riedmiller. Neural Fitted Q Iteration - First Experiences with a Data Efficient Neural Reinforcement Learning Method. *European Conference on Machine Learning (ECML)*, 2005
- M. Hessel et al. Rainbow: combining improvements in deep reinforcement learning. *AAAI Conference on Artificial Intelligence (AAAI)*, 2018.
- H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double Q-learning. *AAAI Conference on Artificial Intelligence (AAAI)*, 2016.